Energy-Efficient D2D Caching with Decentralized Federated Deep Reinforcement Learning

Sanqiu Liu[1], Qiang Li[1], Xiaotian Zhou[2], and Xiaohu Ge[1],
[1]Huazhong University of Science and Technology, 430074 P. R. China,
[2]Institute of Intelligent Communication Technologies, Shandong University, Jinan, China.

***Abstract*—In this paper, a cache-enabled device-to-device (D2D) network is investigated, in which an optimization problem on minimizing the long-term energy consumption in content provision is formulated. To alleviate the single-point-of-failure in conventional federated learning (FL) that relies on a single central server, a novel decentralized federated deep reinforcement learning (DFDRL) framework is proposed to coordinate the collaborative content caching and provisioning among proximal devices. By aggregating the local models among distributed devices, higher robustness is achieved in making caching decisions and lower energy is consumed in provisioning contents. Compared to the state-of-the-art methods, the energy consumption is reduced by up to 8.78%.**

***Index Terms*—Device-to-device networks, collaborative edge caching, energy consumption, decentralized federated learning, deep reinforcement learning.**

I. Introduction

With the widespread use of Internet of Things devices and the rapid growth of mobile internet applications, global mobile data traffic has surged unprecedentedly [1]. To meet the high data rate and low latency requirements of content-centric applications, device-to-device (D2D) caching [2], [3], [4], [5], [6] has been proposed as an effective solution to reduce the redundant traffic and enhance the quality of service (QoS).

Conventional D2D caching approaches primarily operate under the assumption of perfect prior knowledge of content popularity distributions [2], [7]. However, such assumptions often prove inadequate in dynamic environments characterized by continuous content generation and evolving user preferences [8]. Recent advances have introduced reinforcement learning-based caching strategies that leverage historical request patterns [9], demonstrating remarkable potential for sequential decision-making in dynamic settings [10], [11], [12]. On the other hand, responding to the growing privacy concerns in D2D networks, researchers have developed federated learning (FL)-based caching frameworks [14], [13]. These innovative solutions employ model aggregation instead of raw data sharing, effectively enhancing privacy protection while maintaining caching performance.

Current FL-based D2D caching approaches primarily employ a central server (CS) architecture [15], requiring all participating devices to upload their model parameters to the CS for global model maintenance and updates. To minimize the transmission latency, Cheng et al. [13] proposed a double-layer blockchain-based deep reinforcement federated learning based D2D caching scheme, in which an FL with the double-stage cluster-based framework was constructed to train the global model. In [14], to maximize the system benefit per unit cost, a community detection and attention-weighted federated learning based proactive edge caching strategy was proposed in D2D-assisted wireless networks, in which an attention-weighted aggregation mechanism was proposed to generate different aggregation weights for different local quality models in the FL aggregation step. However, these CS-dominated D2D caching frameworks utilized in [13], [14] present several critical limitations, including the vulnerability to single-point-of-failure, excessive resource consumption, and insufficient operational flexibility [16].

Inspired by the aforementioned studies, a cache-enabled D2D network is first constructed in dynamic environment with continuous content generation and evolving user preferences. In order to facilitate the collaborative caching among proximal devices and avoid the single-point-of-failure in the conventional FL, a decentralized federated deep reinforcement learning (DFDRL) framework is then proposed. The main contributions of this paper are summarized as follows:

- In order to enable efficient D2D caching, a hierarchical architecture of content provision is established for the considered cache-enabled D2D network. Specifically, depending on whether a cache hit occurs, the requested content can be fetched from the local cache, from a neighboring device, or from the cloud. To achieve energy-efficient content caching and provisioning in the long term, an optimization problem on minimizing the average system energy consumption is formulated.
- In order to avoid the single-point-of-failure and the deficiency of conventional FL in training a single global model, a DFDRL-based collaborative D2D caching algorithm is proposed. Specifically, each device periodically exchanges model parameters with its neighbors via D2D links to maintain an aggregated model locally, through which higher robustness is achieved in making personalized caching decisions and lower energy is consumed in provisioning the requested contents.
- A real-world dataset from MovieLens is employed to

The authors would like to acknowledge the support from the National Natural Science Foundation of China (NSFC) under grant U24A20212, the Interdisciplinary Research Program of HUST under grant 2024JCYJ022, and the Hubei Provincial Key R&D Program under grant 2022EHB014.

© Copyright IEICE – The 30th Asia-Pacific Conference on Communications (APCC 2025)
Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 06,2026 at 07:04:08 UTC from IEEE Xplore. Restrictions apply.

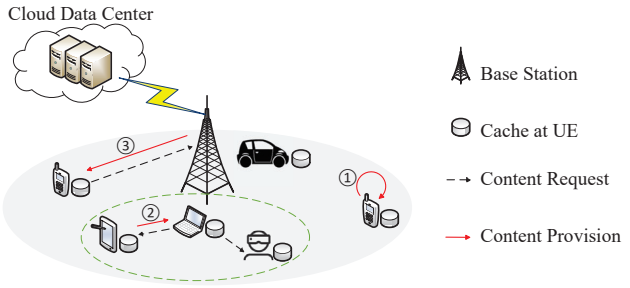


Fig. 1: An illustration of the considered cache-enabled D2D network with hierarchical architecture of content provision, ① local cache hit, ② D2D cache hit, ③ cloud serve upon cache miss.

assess the system energy consumption and cache-hit-ratio (CHR) achieved within the considered D2D network. Compared to the benchmark methods including Random Caching (RC), Least Recently Used (LRU), DQL without collaboration and FDRL with full collaboration, significant performance gains are achieved by the proposed algorithm. To be specific, the CHR is improved by up to 35.98% and the system energy consumption is reduced by up to 8.78%.

II. System Model

A. System Model of Cache-Enabled D2D Network

As shown in Fig. 1, the considered cache-enabled D2D network consists of a user equipment (UE) set $\mathcal{K} = \{1, 2, \cdots, K\}$, which are randomly distributed within the coverage of a base station (BS). The BS, at the location of L_0, is connected to the cloud data center via backhaul to provide services to UEs as required. Without loss of generality, each UE is assumed to have a limited cache storage S_k for $k \in \mathcal{K}$, and request contents with the same size c from a library $\mathcal{F} = \{1, 2, \cdots, F\}$ [13], [14]. Let $L_k \in \mathbb{R}^2$ denote the location of the kth UE, then the set of potential neighbors of the kth UE is denoted as $\mathcal{N}(k) = \{k' | k' \in \mathcal{K}, \|L_k - L_{k'}\| \leq l\}$, where l denotes the D2D communication radius. It is assumed that the BS offers a common communication channel for exchanging control messages, and BS-UE and D2D communications do not interfere with each other [2], [17].

To make efficient caching decisions sequentially in the long term, the system clock is assumed to operate in discrete time slots indexed by $t \in \{1, 2, \cdots, T\}$ [10], [17]. At the beginning of a time slot, a caching decision is made by each UE by learning from the historical request data [14], [17]. Within the duration of a time slot, content requests are generated randomly and sequentially by each UE according to a Poisson process, with arrival rate λ [requests/time slot] [10]. For ease of exposition, the request sequence generated by the kth UE in the tth time slot is defined as $\mathcal{R}(k,t) = \left\{R_1^{k,t}, \cdots, R_n^{k,t}, \cdots, R_{N(k,t)}^{k,t}\right\}$, where $R_n^{k,t} \in \mathcal{F}$ represents the ID of the nth requested content, and $N(k,t)$ indicates the total number of requests generated by the kth UE in the tth time slot.

B. Hierarchical Architecture of Content Provision

As shown in Fig. 1, depending on the location at which the requested content is found, a hierarchical architecture of content provision is established, as detailed in the following.

1) Local Cache Hit: When UE k requests a content f, it first checks its local cache. For ease of exposition, an indicator function $I(f,k) \in \{0,1\}$ is defined to indicate whether the requested content f is cached by the kth UE in the tth time slot. Then the corresponding local CHR for the kth UE in the tth time slot is expressed as

$$h_{\text{self}}(k,t) = \frac{\sum_{f \in \mathcal{R}(k,t)} I(f,k)}{N(k,t)}. \tag{1}$$

Upon a local cache hit, the kth UE will serve itself by directly retrieving content f from its local cache. The corresponding energy consumption and latency are negligible.

2) D2D Cache Hit: If the requested content f is not cached locally, UE k will inquire the neighboring UEs $\mathcal{N}(k)$ within the effective D2D communication radius l. Similarly, we define $I\big(f, \mathcal{N}(k)\big) \in \{0,1\}$ as an indicator function indicating whether the requested content f is cached in the neighboring UEs of the kth UE in the tth time slot. Then the corresponding D2D CHR for the kth UE in the tth time slot is expressed as

$$h_{\text{d2d}}(k,t) = \frac{\sum_{f \in \mathcal{R}(k,t)} \max\Big(0, I\big(f, \mathcal{N}(k)\big) - I(f,k)\Big)}{N(k,t)}. \tag{2}$$

Upon a D2D cache hit at a neighboring UE, the content f is retrieved through D2D communications. Then the corresponding energy consumption in delivering content f from UE k^* to UE k is given by

$$E_{\text{d2d}}(k,t) = \sum_{f \in \mathcal{R}(k,t)} \max\Big(0, I\big(f, \mathcal{N}(k)\big) - I(f,k)\Big) \cdot P_{k^*} \cdot d_{kk^*}^f, \tag{3}$$

where $k^* \in \mathcal{N}(k)$ denotes the nearest UE that can provide the requested content f, P_{k^*} denotes the transmission power of the k^*th UE, and $d_{kk^*}^f$ denotes the corresponding transmission time that is given by

$$d_{kk^*}^f = \frac{c}{B_{kk^*} \cdot \log_2\left(1 + \frac{P_{k^*} \cdot \kappa \cdot \|L_k - L_{k^*}\|^{-\varepsilon}}{\sigma^2}\right)}. \tag{4}$$

Here B_{kk^*} denotes the allocated bandwidth for each D2D link, κ and ε represent the path-loss constant and exponent of D2D links respectively, and σ^2 denotes the additive white Gaussian noise power.

3) Cache Miss: In the event of a cache miss, where the requested content f cannot be found in the local cache nor in neighboring UEs, the BS will fetch content f from the cloud data center and transmit it to the requesting UE. Then the corresponding energy consumption in delivering content

© Copyright IEICE – The 30th Asia-Pacific Conference on Communications (APCC 2025)
Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 06,2026 at 07:04:08 UTC from IEEE Xplore. Restrictions apply.

f from the BS to UE k is given by

$$E_{\text{bs}}(k,t)=\sum_{f\in\mathcal{R}(k,t)}\max\Big(0,1-I\big(f,\mathcal{N}(k)\big)-I(f,k)\Big)\cdot(P_0\cdot d_{k0}^{f}+E_0), \tag{5}$$

where E_0 is a constant that denotes the energy consumption for BS to fetch a content from the cloud, P_0 denotes the transmission power of BS, and d_{k0}^{f} denotes the corresponding transmission time that is given by

$$d_{k0}^{f}=\frac{c}{B_{k0}\cdot\log_2\left(1+\frac{P_0\cdot\zeta\cdot\|L_k-L_0\|^{-\nu}}{\sigma^2}\right)}. \tag{6}$$

Here B_{k0} denotes the allocated bandwidth for each BS-UE link, and ζ and ν represent the path-loss constant and exponent of BS-UE links, respectively.

C. Content Replacement Energy Consumption

At the beginning of each time slot, content replacement is executed according to the caching decisions, where the contents to be replaced are downloaded by the UEs from the cloud [18]. For ease of exposition, we let $V_t^k \in \{0,1\}$ denote whether the kth UE replaces a cached content at the beginning of the tth time slot. Then the corresponding energy consumption in content replacement at the kth UE in the tth time slot can be expressed as

$$E_{\text{rep}}(k,t)=V_t^k\cdot(P_0\cdot d_{k0}^{f}+E_0). \tag{7}$$

III. Proposed DFDRL-Based Collaborative D2D Caching Algorithm

A. Problem Formulation

Based on the above definitions and analysis, an optimization problem on minimizing the long-term average system energy consumption in content provisioning is formulated as

$$\begin{aligned}\min_{\left\{I(f,k),I\left(f,\mathcal{N}(k)\right)\right\}}\ &\frac{1}{T}\sum_{t=1}^{T}\sum_{k=1}^{K}\left[E_{\text{d2d}}(k,t)+E_{\text{bs}}(k,t)+E_{\text{rep}}(k,t)\right],\\ s.t.\quad &\text{C}_1: I(f,k)\in\{0,1\},\\ &\text{C}_2: I\big(f,\mathcal{N}(k)\big)\in\{0,1\},\\ &\text{C}_3: V_t^k\in\{0,1\},\\ &\text{C}_4: \sum_{f\in\mathcal{F}} I(f,k)\cdot c\le S_k,\forall\ k,\\ &\text{C}_5: d_{kk^*}^{f}\le d_{k0}^{f}.\end{aligned} \tag{8}$$

Here C_1, C_2 and C_3 denote the binary indicator function; C_4 guarantees that the total amount of cached contents does not exceed the cache storage of each UE [8]; C_5 ensures that the D2D content delivery is allowed only when it achieves a lower latency than fetching a content from the cloud. A close look into (8) reveals that the objective function corresponds to a challenging integer nonlinear programming problem, which falls into the NP-hard category. Next a DFDRL-based collaborative D2D caching algorithm is proposed to address the formulated problem.

Algorithm 1 Proposed DFDRL-Based Collaborative D2D Caching Algorithm.

```
Initialize EvaluateNet and TargetNet.
for time slot t = 1, 2, ..., T do
    for UE k = 1, 2, ..., K in parallel do
        Observe state s_t^k.
        Select an action a_t^k by arg max_A Q(s_t^k, A).
        Observe the reward r_t^k and the new state s_{t+1}^k.
        Store {s_t^k, a_t^k, r_t^k, s_{t+1}^k} in D.
        Randomly sample mini-batch {s_j^k, a_j^k, r_j^k, s_{j+1}^k}.
        Update EvaluateNet by minimizing L(w_t^k) in (10).
        if a period ends then
            Exchange local model with neighboring UEs.
            Obtain aggregated model by executing (11).
            Update TargetNet by letting ŵ_t^k = w_t^k.
        end if
    end for
end for
```

B. Proposed DFDRL-Based Collaborative D2D Caching Algorithm

In order to make wise caching decisions collaboratively at proximal UEs, next a DFDRL-based collaborative D2D caching algorithm is proposed. In order to facilitate the effective convergence of the model, it is essential for the local models to undergo a certain number of training iterations before collaborative learning is executed among neighboring UEs. This corresponds to a 'period' for collaborative learning, with each period comprising m time slots.

1) Local Deep Q Learning: In order to make real-time adjustment to the caching strategies at each UE, local deep Q learning (DQL) model training is carried out on a slot basis by utilizing the request sequence $\mathcal{R}(k,t)$. Specifically, during time slot t, the kth UE collects the cache state s_t^k, which is a 0-1 binary vector of F elements and the sum of the elements is $\lfloor\frac{S_k}{c}\rfloor$. With the observation of state s_k^t, an action $a_t^k\in\mathcal{A}=\{0,1,\cdots,\lfloor\frac{S_k}{c}\rfloor\}$ is selected, where $a_t^k=0$ means no cache replacement is needed and $a_t^k\in\{1,\cdots,\lfloor\frac{S_k}{c}\rfloor\}$ means that the cached content at the corresponding location in the cache space will be replaced. Upon executing action a_t^k, the corresponding reward $r_t^k=e^{-[E_{\text{d2d}}(k,t)+E_{\text{bs}}(k,t)+E_{\text{rep}}(k,t)]}$ is obtained, accompanied by a state transition to s_{t+1}^k. Thus, the action-value function can be updated as

$$\begin{aligned}Q(s_{t+1}^k,a_{t+1}^k)=Q(s_t^k,a_t^k)+\alpha\Big[r_t^k+\gamma\max_{a_{t+1}^k}\ \hat{Q}(s_{t+1}^k,a_{t+1}^k)\\ -Q(s_t^k,a_t^k)\Big],\end{aligned} \tag{9}$$

where α denotes the learning rate, γ denotes the discount factor, $Q(s_t^k,a_t^k)$ denotes the evaluation network (EvaluateNet), and $\hat{Q}(s_{t+1}^k,a_{t+1}^k)$ denotes the target network (TargetNet).

DQL model interacts with the environment and updates its parameters through the back propagation of the loss function. Both EvaluateNet and TargetNet are constructed as neural networks, sharing identical architectures while main-

© Copyright IEICE – The 30th Asia-Pacific Conference on Communications (APCC 2025)
Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 06,2026 at 07:04:08 UTC from IEEE Xplore. Restrictions apply.

TABLE I: Default Parameter Configurations

Parameter	Value	Parameter	Value	Parameter	Value
K	40	l	30m	F	3952
c	1Mbits	T	1000	λ	20
$S_k\ \forall\ k$	100Mbits	σ^2	10^{-9}W	$P_{k^*}\ \forall\ k^*$	0.25W
P_0	40W	κ, ζ	10^{-2}	ε, ν	4
B_{kk^*}, B_{k0}	10MHz	m	50	Ω	5000
ϕ	64	α	0.05	γ	0.9

taining distinct parameter sets. The input of EvaluateNet is the current caching state s_t^k and the output is an action $a_t^k = \arg\max_{\mathcal{A}} Q(s_t^k, \mathcal{A})$. A transition $\{s_t^k, a_t^k, r_t^k, s_{t+1}^k\}$ can be constructed and stored in the replay buffer $\mathcal{D}$ with capacity Ω. In the local training phase, a mini-batch of transitions $\{s_j^k, a_j^k, r_j^k, s_{j+1}^k\}$ are randomly sampled from the replay buffer to update EvaluateNet, where $j \in \{1, \cdots, \phi\}$ and ϕ denotes the mini-batch size. Then the output of EvaluateNet $Q(s_j^k, a_j^k; \boldsymbol{w}_t^k)$ and the output of TargetNet $\hat{Q}(s_{j+1}^k, a_{j+1}^k; \hat{\boldsymbol{w}}_t^k)$ can be obtained, where $\boldsymbol{w}_t^k$, $\hat{\boldsymbol{w}}_t^k$ represent the parameters of EvaluateNet and TargetNet, respectively. Then the loss function can be defined as

$$L(\boldsymbol{w}_t^k) = \mathbb{E}\left[\left(r_j^k + \gamma \max_{a_{j+1}^k} \hat{Q}(s_{j+1}^k, a_{j+1}^k; \hat{\boldsymbol{w}}_t^k) - Q(s_j^k, a_j^k; \boldsymbol{w}_t^k)\right)^2\right]. \tag{10}$$

According to the above definitions, gradient descent is applied to (10) to update EvaluateNet. For the sake of simplicity, TargetNet is updated at the beginning of training for each UE in every period by letting $\hat{\boldsymbol{w}}_t^k = \boldsymbol{w}_t^k$ accordingly.

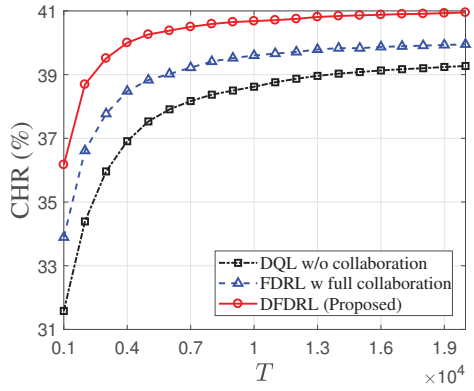
2) Decentralized Federated Aggregation: When a period ends, collaborative learning is activated among neighboring UEs. That is, each UE exchanges its parameter set with the neighboring UEs via D2D links, based on which an aggregated model is constructed at each UE to capture the user preferences in the neighborhood. Specifically, in order to account for the differences in the DQL models of different UEs, different aggregation weights are used, where the aggregated model $\boldsymbol{W}(k, t+1)$ at UE k is given by

$$\boldsymbol{W}(k, t+1) = \sum_{u \in \mathcal{N}(k) \cup \{k\}} H(u, t) \boldsymbol{W}(u, t), \tag{11}$$

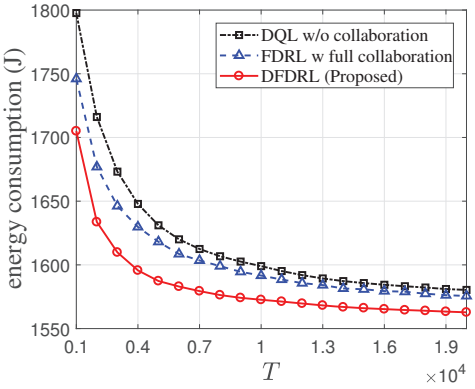
where $\boldsymbol{W}(u, t)$ denotes the trained local model of the uth UE at the end of time slot t, and $H(u, t)$ denotes the aggregation weight of the uth UE in the tth time slot that is given by

$$H(u, t) = \frac{\sum_{t'=t-m+1}^{t} N(u, t')}{\sum_{t'=t-m+1}^{t} \sum_{u \in \mathcal{N}(k) \cup \{k\}} N(u, t')}. \tag{12}$$

It is worth noting that according to this aggregation method, each UE aggregates with its neighbors within the D2D communication range to generate a unique model locally, which to some extent reflects the user preferences of the neighborhood. The details of the proposed DFDRL-based collaborative D2D caching algorithm are presented in Algorithm 1.



(a)



(b)

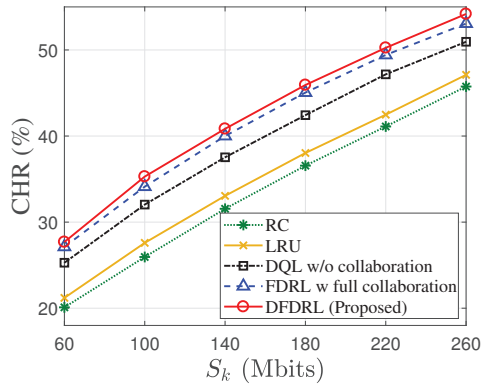
Fig. 2: Comparisons of convergence performance.

Remark 1: The total computational complexities are given by $\mathcal{O}\big(M + m\sum_{k\in\mathcal{K}}(\Omega + \phi YU)\big)$ for the conventional FL and $\mathcal{O}\big(KM + m\sum_{k\in\mathcal{K}}(\Omega + \phi YU)\big)$ for the proposed DFDRL, where M denotes the complexity for one aggregation process [15], Y is the total number of layers and U is the number of neurons per layer [8]. Thus, although the proposed algorithm achieves a slightly higher complexity than the conventional FL, the latter is susceptible to single-point-of-failure. In contrast, by employing a distributed aggregation strategy to capture the user preferences in the neighborhood, the proposed DFDRL-based collaborative D2D caching algorithm achieves a higher robustness in making more personalized caching decisions.

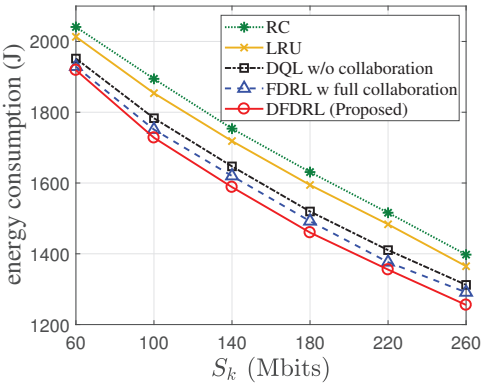
IV. Simulation Results and Discussions

To evaluate the performance of the proposed DFDRL-based D2D collaborative caching algorithm, the MovieLens1M dataset from MovieLens [19] is utilized to simulate content requests in a D2D network. Given that users only rate movies after watching them, it is assumed that a user's rating represents a content request [8]. Without loss of generality, a single macrocell is considered, within which $K = 40$ UEs are randomly distributed and the cache storage of each UE is set to $S_k = 100$Mbits, $\forall k$. The default parameter

© Copyright IEICE – The 30th Asia-Pacific Conference on Communications (APCC 2025)
Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 06,2026 at 07:04:08 UTC from IEEE Xplore. Restrictions apply.

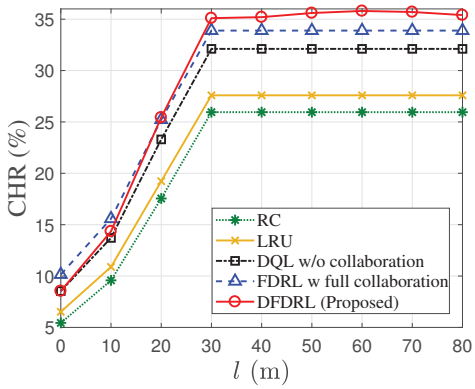


(a)

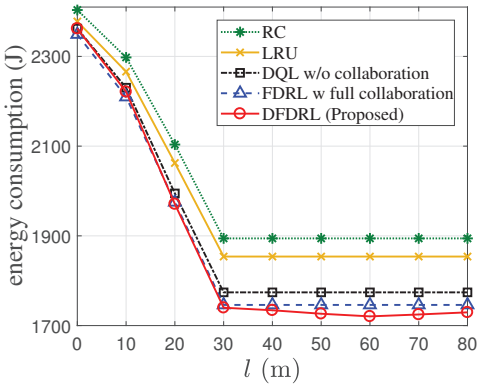


(b)

Fig. 3: The effect of S_k on CHR and system energy consumption.



(a)



(b)

Fig. 4: The effect of l on CHR and system energy consumption.

configurations used in the simulations are listed in Table I. For comparison purposes, existing caching strategies, including FL-based caching with DQL performed at each UE (FDRL with full collaboration) [15], DQL-based caching at each UE with no collaboration (DQL without collaboration) [8], LRU [15], and RC [18], are also simulated.

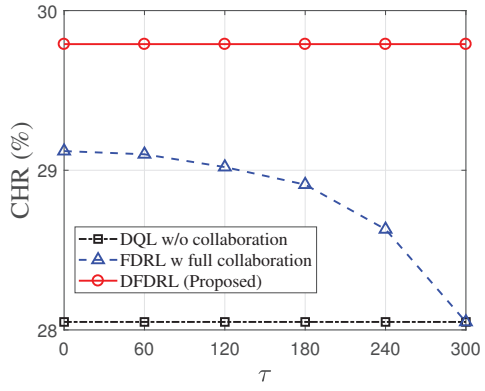
As shown in Fig. 2, the CHR and the system energy consumption under various learning-based methods are presented with respect to the total number of time slots T. It can be observed that as T increases, both the CHR and the energy consumption gradually converge. Furthermore, the proposed DFDRL algorithm always outperforms the other algorithms in terms of both CHR and energy consumption.

As illustrated in Fig. 3, the CHR and the system energy consumption are demonstrated with respect to the cache storage S_k. It is observed that the overall performance improves with increasing S_k. This is reasonable because a larger S_k allows for more contents to be potentially stored in the UEs' cache, enabling UEs to fetch contents with lower energy consumption and higher CHR in the neighborhood. At $S_k = 100$Mbits, compared to RC, LRU, DQL without collaboration and FDRL with full collaboration, it is observed that the CHR is improved by 35.98%, 27.9%, 10.15% and 3.43%, respectively; and the system energy consumption is reduced by 8.78%, 6.79%, 3.09% and 1.32%, respectively.

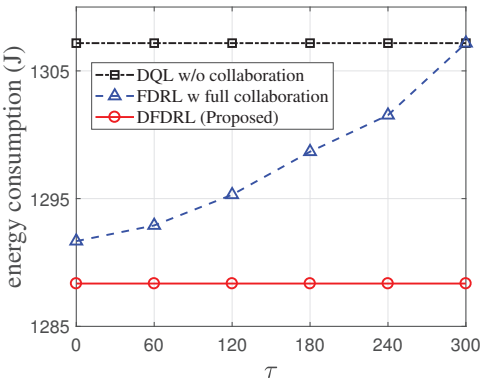
From the above observations, it is worth noting that learning-based methods outperform conventional caching algorithms like RC and LRU by leveraging the contextual and historical data. While FDRL with full collaboration surpasses DQL without collaboration, the proposed DFDRL algorithm achieves the highest CHR and lowest energy consumption. Unlike FDRL with full collaboration that maintains a single global model at the BS that fails to reflect the user preferences at different locations, the proposed DFDRL algorithm maintains a unique personalized model at each UE locally by federated aggregation among nearby UEs, thus outperforming the baselines.

As shown in Fig. 4, when $l \leq 30$m, CHR is improved and energy consumption is reduced with increasing l, as more neighbors enable more efficient content provision in the proximity. When $l > 30$m, metrics stabilize in that the D2D content delivery requires a higher latency that violates constraint C_5 in (8), then it is preferred to fetch the requested content from the cloud. Moreover, when $l < 20$m, it is observed that FDRL with full collaboration outperforms the proposed DFDRL algorithm. This is reasonable because the

© Copyright IEICE – The 30th Asia-Pacific Conference on Communications (APCC 2025)
Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 06,2026 at 07:04:08 UTC from IEEE Xplore. Restrictions apply.



(a)



(b)

Fig. 5: CHR and system energy consumption with varying number of CS failure time slots.

performance of proposed DFDRL algorithm is limited by the insufficient number of neighbors. Thus, an appropriate D2D communication radius l needs to be selected to make full use of the proposed DFDRL algorithm's advantages.

Additionally, the impact of single-point-of-failure is also illustrated in Fig. 5, where a total number of $T = 300$ time slots is considered, within which the CS fails during a number of τ time slots. That is, when the CS breaks down, there is no way to perform centralized model aggregation to maintain a global model. Thus the FL with full collaboration degrades to the DQL without collaboration, where each UE trains a DQL model locally with no collaboration. This can be observed in Fig. 5, where the CHR and energy consumption of the FDRL with full collaboration gradually approach that of the DQL without collaboration as τ increases. In contrast, higher robustness is achieved and lower energy is consumed by the proposed DFDRL algorithm owing to the decentralized training process and collaborative content provisioning among neighboring devices.

V. Conclusions

In this paper, collaborative caching was investigated in a cache-enabled D2D network. In view of the single-point-of-failure and the deficiency of the conventional FL in training a single model, a novel DFDRL-based collaborative D2D caching algorithm was proposed. By periodically aggregating models among neighboring devices, higher robustness was achieved in making personalized caching decisions and lower energy was consumed in provisioning the requested contents. Simulation results demonstrated that a better caching performance was achieved by the proposed algorithm as compared to the state-of-the-art methods, where the CHR was improved by up to 35.98% and the system energy consumption was reduced by up to 8.78%, respectively.

References

[1] X. Gao, et al., "A stochastic geometry-based cost-efficient caching strategy in device-to-device-enabled multi-layer heterogeneous networks," *IEEE Trans. Veh. Technol.*, vol. 73, no. 10, pp. 14 749–14 762, 2024.

[2] Q. Li, et al., "D2d-assisted caching on truncated zipf distribution," *IEEE Access*, vol. 7, pp. 13 411–13 421, 2019.

[3] Q. Li, et al., "Edge caching in wireless infostation networks: Deployment and cache content placement," in *IEEE INFOCOM*, 2019, pp. 1–6.

[4] X. Wu, et al., "D2d assisted coded caching design for multi-access networks," *IEEE Wirel. Commun. Lett.*, vol. 13, pp. 2702–2706, 2024.

[5] X. Li, et al., "Hierarchical edge caching in device-to-device aided mobile networks: Modeling, optimization, and design," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 8, pp. 1768–1785, 2018.

[6] A. Powari, et al., "Sum rate maximization for noma with simultaneous cache-enabled d2d communications," *IEEE Wirel. Commun. Lett.*, vol. 14, no. 2, pp. 479–483, 2025.

[7] S. Han, et al., "Data-supported caching policy optimization for wireless d2d caching networks," *IEEE Trans. Commun.*, vol. 69, no. 11, pp. 7618–7630, 2021.

[8] S. Liu, et al., "Personalized collaborative edge caching with federated transfer deep reinforcement learning," *IEEE Commun. Lett.*, vol. 28, no. 9, pp. 2096–2100, 2024.

[9] R. Peng, et al., "Collaborative content caching algorithm for large-scale istns based on mappo," *IEEE Wirel. Commun. Lett.*, vol. 13, no. 11, pp. 3069–3073, 2024.

[10] W. Jiang, et al., "Multi-agent reinforcement learning for efficient content caching in mobile d2d networks," *IEEE Trans. Wirel. Commun.*, vol. 18, no. 3, pp. 1610–1622, 2019.

[11] L. Li, et al., "Deep reinforcement learning approaches for content caching in cache-enabled d2d networks," *IEEE Internet Things J.*, vol. 7, no. 1, pp. 544–557, 2020.

[12] Y. Bai, et al., "A deep-reinforcement-learning-based social-aware cooperative caching scheme in d2d communication networks," *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9634–9645, 2023.

[13] R. Cheng, et al., "Blockchain-empowered federated learning approach for an intelligent and reliable d2d caching scheme," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 7879–7890, 2022.

[14] D. Li, et al., "Community detection and attention-weighted federated learning based proactive edge caching for d2d-assisted wireless networks," *IEEE Trans. Wirel. Commun.*, vol. 22, pp. 7287–7303, 2023.

[15] M. Lei, et al., "Partially collaborative edge caching based on federated deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1389–1394, 2023.

[16] S. Chen, et al., "Enhancing decentralized and personalized federated learning with topology construction," *IEEE. Trans. Mob. Comput.*, vol. 23, no. 10, pp. 9692–9707, 2024.

[17] Y. Yan, et al., "Cooperative caching and fetching in d2d communications - a fully decentralized multi-agent reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 16 095–16 109, 2020.

[18] M. Lei, et al., "Deep deterministic policy gradient-based edge caching: An inherent performance tradeoff," in *IEEE GLOBECOM*, 2021, pp. 1–7.

[19] F. M. Harper, et al., "The movielens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, pp. 19:1–19:19, 2016. [Online]. Available: https://api.semanticscholar.org/CorpusID:16619709

© Copyright IEICE – The 30th Asia-Pacific Conference on Communications (APCC 2025)
Authorized licensed use limited to: Shaanxi Normal University. Downloaded on June 06,2026 at 07:04:08 UTC from IEEE Xplore. Restrictions apply.